

Guia de Arguição Completo: Alinhamento com a Checklist do Barema

Este documento apresenta perguntas e respostas mapeadas **exatamente na mesma estrutura e ordem dos critérios do Barema de Avaliação do Professor** para o Problema 3. Use-o como guia rápido para estudar e conduzir a demonstração de 30 minutos no laboratório.

1. ARQUITETURA (TOLERÂNCIA A FALHAS E DESCENTRALIZAÇÃO)

[] Item 1.1: Múltiplos nós independentes com cópias individuais do ledger

Pergunta do Professor: "Como posso verificar se cada nó tem sua própria cópia local do banco de dados e não está compartilhando uma base de dados centralizada?"

Resposta: "Cada Broker em execução é um processo independente que salva sua própria cadeia de blocos em um arquivo JSON local chamado `chain_<broker_id>.json` (ex: `chain_B1.json`, `chain_B2.json`, etc.) na pasta raiz de execução. Quando executado em máquinas separadas (via `subir_distribuido.sh`), esses arquivos são gravados nos HDs locais de cada máquina e sincronizados apenas via mensagens P2P pela rede."

Onde está no código: Funções `Save` e `Load` em [chain.go](#).

[] Item 1.2: Ponto único de falha ou autoridade central disfarçada

Pergunta do Professor: "Existe algum nó mestre oculto ou servidor de saldos que, se cair, derruba o sistema?"

Resposta: "Não. Não há autoridade central. Os saldos são derivados por cada nó localmente a partir de sua própria cópia da blockchain. Embora o Broker `B1` sirva de bootstrap para a descoberta de peers no início da simulação, uma vez que a malha P2P é estabelecida, `B1` pode cair e os brokers sobreviventes (`B2`, `B3`, `B4`) continuam minerando blocos, validando transações e despachando drones autonomamente."

Onde está no código: A malha P2P é mantida pelas conexões em `b.vizinhos` em [broker_main.go](#).

[] Item 1.3: Resiliência prática (derrubar nós na demonstração)

Pergunta do Professor: "Vou derrubar o Broker 1. O sistema continua operando?"

Resposta: "Sim. Pode derrubar. O anel lógico detectará a queda do nó vizinho através do timeout de batimentos cardíacos (Heartbeats) e redistribuirá o monitoramento do setor de **B1** para o vizinho ativo mais próximo. Novos alertas do setor de **B1** serão capturados e os drones serão despachados pelo vizinho."

Onde está no código: Detecção em **loopDetectarFalhas** e re-roteamento em **responsavelPorSetor** em [broker_main.go](#).

[] Item 1.4: Escolha da tecnologia e trade-offs

Pergunta do Professor: "Por que implementar uma blockchain do zero em vez de usar Ethereum ou Hyperledger Fabric? Quais os trade-offs?"

Resposta: "Optamos por implementar a blockchain do zero em Go para demonstrar domínio absoluto do algoritmo de consenso e controle concorrente. Os trade-offs são:

- **Prós:** Extrema leveza computacional (roda 4 nós simultâneos e clientes na mesma máquina no laboratório), inicialização instantânea, e ausência de oráculos complexos de IoT.
- **Contras:** Não possui sandbox para smart contracts dinâmicos (regras são fixas no Go) e a segurança de rede é restrita ao ambiente controlado do consórcio (4 validadores)."

Onde consultar: A comparação detalhada está na Tabela comparativa da Pergunta 6 em [perguntas_respostas_barema.md](#).

2. COMUNICAÇÃO (REDE P2P E GOSSIP)

[] Item 2.1: Conectividade de nós e protocolo de mensagens

Pergunta do Professor: "Como os brokers se conectam e trocam informações?"

Resposta: "Eles se conectam usando conexões TCP brutas em sockets persistentes. O protocolo de mensagens é estruturado com payload JSON (**models.MensagemBroker**). Drones e navios conversam com o broker do seu setor usando requisições HTTP REST convencionais."

Onde está no código: Estrutura de mensagens em [dados.go](#).

[] Item 2.2: Propagação de blocos e transações

Pergunta do Professor: "Como novas transações e blocos atingem toda a rede?"

Resposta: "Utiliza-se propagação Gossip. Quando um nó recebe uma transação via API, ele a valida localmente e a envia via `MsgTxBroadcast` para seus vizinhos imediatos. O mesmo ocorre quando um bloco é proposto (`MsgBlockProposal`) ou confirmado (`MsgBlockCommit`)."

Onde está no código: Função `broadcastVizinhos` em [broker_main.go](#).

[] Item 2.3: Mecanismo de Consenso (Proof of Authority - PoA)

Pergunta do Professor: "Qual o consenso utilizado e como funciona tecnicamente?"

Resposta: "Adotamos o **Proof of Authority (PoA) Round-Robin**. O proponente do bloco de índice `N` é `N % 4`. O proponente monta o bloco, assina e envia para votação. Ao coletar pelo menos **3 assinaturas válidas de validadores conhecidos (quórum de 3/4)**, o bloco é commitado on-chain."

Onde está no código: Regras em [consensus.go](#) e processamento em `loopConsensus` em [broker_main.go](#).

[] Item 2.4: Resolução de conflitos e forks

Pergunta do Professor: "Como o sistema lida com divergências ou forks?"

Resposta: "Forks são impossíveis on-chain no fluxo normal porque apenas o proponente da rodada pode submeter o bloco na altura `N` e o quórum de 3/4 de validadores garante finalidade imediata. Para nós que caem e ficam atrasados, a sincronização é feita de forma tardia (nó pede a chain mais longa via `MsgChainSync` e adota a do vizinho)."

Onde está no código: Caso `MsgChainResponse` em [broker_main.go](#).

[] Item 2.5: Correlação com o Problema anterior (Exclusão Mútua vs Blockchain)

Pergunta do Professor: "Por que você manteve o algoritmo de exclusão mútua do Problema 2 se já tem uma blockchain distribuída?"

Resposta: "Eles atuam em escalas de tempo distintas. O consenso da blockchain leva segundos para fechar um bloco (neste caso, a cada 10s). A decolagem e coordenação física do drone precisa de respostas em milissegundos. Usamos **exclusão mútua e Lamport** para decidir fisicamente quem atende a ocorrência imediatamente e a **blockchain** para o registro financeiro e auditoria imutável posterior."

Onde consultar: Detalhes na Pergunta 8 do documento [perguntas_respostas_barema.md](#).

3. GESTÃO DE ATIVOS (TOKENS E SALDOS)

[] Item 3.1: Emissão e Recarga de Créditos (Tokens ELIS)

Pergunta do Professor: "Como os créditos das empresas são criados?"

Resposta: "Os créditos iniciam com um `TxMint` de 1000.0 ELIS para cada empresa registrada no bloco Gênese. Adicionalmente, há um faucet periódico (`ENABLE_TOKEN_REPLENISHMENT`) controlado por variáveis de ambiente que mintam 50.0 ELIS a cada 60s para as empresas registradas na rede."

Onde está no código: Inicialização em `Genesis()` em [chain.go](#) e reabastecimento em `loopReplenishment` em [broker_main.go](#).

[] Item 3.2: Saldo derivado do histórico (On-chain state)

Pergunta do Professor: "O saldo das empresas fica em algum banco de dados separado?"

Resposta: "Não. Os saldos são computados sob demanda percorrendo o histórico de transações confirmadas nos blocos da blockchain. O estado de saldo é volátil e recalculado na inicialização e a cada novo bloco minerado."

Onde está no código: Função `RebuildState` em [chain.go](#).

[] Item 3.3: Transferências Autenticadas por Assinatura Digital

Pergunta do Professor: "Como garantem que apenas a empresa dona dos créditos pode transferi-los?"

Resposta: "Toda transação que altera saldos deve conter a assinatura digital do remetente, gerada a partir da sua chave privada ECDSA secp256k1. O broker verifica se a assinatura confere com a chave pública do campo `From` antes de aceitar a transação na mempool."

Onde está no código: Verificação em `VerifyTxSignature` em [chain.go](#).

[] Item 3.4: Demonstração de transferência entre nós distintos

Pergunta do Professor: "Faça uma transferência de créditos de um nó e mostre a confirmação em outro."

Resposta: "Podemos transferir créditos usando o CLI (`./hormuz_cli transfer <para> <quantidade> <semente_da_empresa>`). A transação é enviada para um broker, incluída em bloco, e após consenso, o novo saldo atualizado pode ser visualizado acessório à API ou ao monitor web rodando em qualquer outra máquina conectada."

4. PREVENÇÃO DE DUPLO GASTO

[] Item 4.1: Teste prático de duplo gasto (Requisições concorrentes com o mesmo saldo)

Pergunta do Professor: "Execute o teste prático de duplo gasto agora."

Resposta: "Podemos demonstrar submetendo duas transações concorrentes que, somadas, estouram o saldo da empresa. O sistema aceitará a primeira transação na mempool, mas rejeitará a segunda imediatamente com erro de saldo insuficiente (**insufficient balance**), provando a proteção em tempo real."

[] Item 4.2: Validação contra o estado do ledger (Mempool e Bloco)

Pergunta do Professor: "A validação do saldo é feita na interface do usuário ou no motor da blockchain?"

Resposta: "No motor da blockchain. O broker calcula o saldo on-chain atualizado da empresa e desconta as transações pendentes dela na mempool antes de permitir qualquer inserção."

Onde está no código: Função **AddTxToMempool** em [chain.go](#).

[] Item 4.3: Momento de detecção no fluxo

Pergunta do Professor: "Em que ponto o duplo gasto é bloqueado?"

Resposta: "Ele é bloqueado em três fases:

1. **Na Mempool (Fase P2P):** Rejeitado no recebimento da transação de forma local.
2. **Na Proposta do Bloco (Consenso):** Brokers executam **ValidateBlockTransactions** no bloco proposto; se houver duplo gasto concorrente empacotado, a proposta é descartada.
3. **No Commit (Gravação):** O ledger re-valida o bloco antes de gravar em disco."

Onde consultar: Detalhes na Pergunta 5 de [perguntas_respostas_barema.md](#).

5. REQUISIÇÃO E PAGAMENTO DE ESCOLTAS

[] Item 5.1: Despacho condicionado ao pagamento prévio

Pergunta do Professor: "O drone decola antes ou depois do pagamento?"

Resposta: "Decola apenas **depois** da confirmação do pagamento. O navio detecta a ocorrência em **AGUARDANDO_PAGAMENTO**, paga, e o broker só insere a ocorrência na fila de prioridades do drone quando o bloco contendo o pagamento é commitado on-chain no status **PAGO**."

Onde está no código: Função **processarTransacoesBloco** em [broker_main.go](#).

[] Item 5.2: Comportamento sem saldo (0 ELIS)

Pergunta do Professor: "O que acontece se uma empresa tentar chamar a escolta sem ter dinheiro?"

Resposta: "A transação de pagamento é rejeitada pelo broker. O status da ocorrência permanece eternamente em **AGUARDANDO_PAGAMENTO** e o drone nunca decola. (Ver roteiro de teste prático na Pergunta 12 do FAQ)."

Onde está no código: Tratamento em **pagarOcorrencia** em [broker_main.go](#).

[] Item 5.3: Teste de Concorrência de drones

Pergunta do Professor: "Se duas companhias solicitarem o mesmo drone ao mesmo tempo, o drone é alocado em dobro?"

Resposta: "Não. O broker faz o despacho verificando a disponibilidade do drone com exclusão mútua (**d.Disponivel()**). O drone é bloqueado para novas missões imediatamente no momento do despacho."

Onde está no código: Função **tentarDespachar** em [broker_main.go](#).

[] Item 5.4: Transação de pagamento registrada no ledger

Pergunta do Professor: "Onde vejo que o pagamento da escolta foi gravado na blockchain?"

Resposta: "Na aba de transações e na aba **PAGAMENTOS** do painel web. Ele lista a transação com ID do bloco correspondente, o valor pago e o navio."

Onde consultar: APIs expostas pelo monitor em [monitor_main.go](#).

6. LOG DE OPERAÇÕES IMUTÁVEL (LAUDOS)

[] Item 6.1: Laudo pós-missão com dados da escolta

Pergunta do Professor: "Como e onde o laudo do drone é gravado?"

Resposta: "Ao terminar o trajeto físico, o drone reporta a telemetria final. O broker monta uma transação **MISSION_LOG** (**TxMissionLog**) contendo o laudo com dados do drone, navio, status final e data/hora, que é minerada no bloco subsequente."

Onde está no código: Registro em **processarMensagemDrone** em [broker_main.go](#).

[] Item 6.2: Teste de adulteração manual de registros

Pergunta do Professor: "Se eu alterar um registro no arquivo JSON local, o que acontece?"

Resposta: "Ao alterar manualmente uma letra do laudo no arquivo local de um Broker (ex: `chain_B1.json`), o hash daquele bloco muda. A cadeia de blocos subsequente quebra (pois o `prev_hash` não bate) e as assinaturas dos validadores falham na verificação. Os vizinhos detectarão a invalidez e isolarão o nó alterado."

Onde está no código: Verificação do hash e hashes anteriores em `AddBlock` em [chain.go](#).

[] Item 6.3: O que garante a imutabilidade?

Pergunta do Professor: "Quais os pilares matemáticos que garantem que ninguém pode apagar um laudo?"

Resposta: "A criptografia de hashing (SHA-256) ligando de forma inseparável o bloco atual ao bloco anterior (`prev_hash`), combinada com as assinaturas digitais ECDSA dos validadores em consenso PoA (quórum 3/4) gravadas no cabeçalho do bloco."

7. TRANSPARÊNCIA E AUDITABILIDADE

[] Item 7.1: Consulta pública sem privilégios (Interface e API)

Pergunta do Professor: "Como auditar as transações e laudos sem permissões administrativas?"

Resposta: "Qualquer usuário do consórcio pode abrir o Painel Web (Monitor) ou usar chamadas GET nas APIs abertas do broker (ex: `/api/explorer/laudos`). Para a privacidade dos dados, os payloads sensíveis são mascarados como **CONFIDENCIAL**, mas podem ser descriptografados digitando a senha universal de auditoria da simulação (`1234`)."

Onde consultar: Função `revelarLaudo` em [monitor_main.go](#).

[] Item 7.2: Consistência entre nós distintos

Pergunta do Professor: "Se eu consultar o bloco 5 no Broker 2 e no Broker 4, o dado é idêntico?"

Resposta: "Sim, pois a blockchain é idêntica e replicada em ambos os nós. A API de qualquer broker ativo retornará exatamente os mesmos blocos e hashes."

[] Item 7.3: Rastreamento do histórico completo (Auditoria)

Pergunta do Professor: "Como rastrear a origem dos créditos de uma empresa ou todas as missões de um drone?"

Resposta: "Através do explorer de blocos na interface web ou consultando o endpoint de pagamentos `/api/explorer/payments?company=...`. Por lá, é possível ver cada transação de **MINT** que gerou créditos e cada escolta paga associada."

8. DOCUMENTAÇÃO

[] Item 8.1: README claro e código comentado no GitHub

Pergunta do Professor: "O projeto possui documentação de execução para múltiplos nós e comentários explicativos?"

Resposta: "Sim. O [README.md](#) detalha todos os pré-requisitos, instruções de instalação dos nós no laboratório de forma centralizada ou distribuída (4 PCs), e o código-fonte foi integralmente comentado em português para facilitar a auditoria."